

Sequential RV64I RISC-V Processor Design

Introduction to Processor Architecture Project Report

Naga Rama Hari Kumar (2024102045)
Chandini Gayathri (2024102020)
Veekshita Sai (2024102026)
IIT Hyderabad
Spring 2026

ABSTRACT

This project focuses on the design and implementation of a 64-bit single-cycle RISC-V processor using Verilog. The processor supports a basic subset of RV64I instructions such as arithmetic, logical, load, store, and branch operations. In a single-cycle architecture, every instruction completes all five stages fetch, decode, execute, memory access, and write back within one clock cycle.

The aim of this phase was to build a functionally correct and modular processor that clearly demonstrates how datapath and control work together. The design was verified through simulations and multiple test cases to ensure correct execution. This implementation forms the base for future extensions such as pipelining.

I. INTRODUCTION

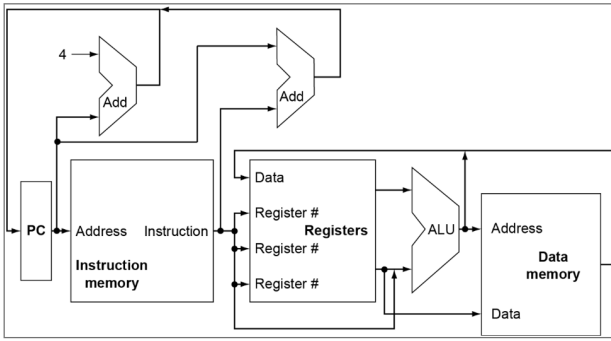


Fig. 1.

RISC-V is a simple and flexible instruction set architecture that is widely used for learning processor design. In this project, we implemented a 64-bit single-cycle RISC-V processor to understand how instructions are executed at the hardware level.

We started with a sequential (single-cycle) design because it is easier to understand and debug. In this model, each instruction finishes completely before the next one begins. There is no overlap between instructions, which means there are no hazards. However, the clock period must be long enough for the slowest instruction (usually a load), which limits performance.

The processor supports instructions such as add, sub, and, or, addi, ld, sd, and beq. The architecture follows the standard five stages: Instruction Fetch, Decode, Execute, Memory Access, and Write Back. Each module like the Program Counter, Register File, ALU, Control Unit, and Memory was implemented separately and then integrated.

This sequential processor helps us clearly understand datapath flow and control logic, and it serves as a strong foundation for building a pipelined processor in the next phase.

II. BACKGROUND AND DESIGN PRINCIPLES

This processor is based on the single-cycle datapath model. In a single-cycle design, every instruction completes all five stages fetch, decode, execute, memory access, and write-back within one clock cycle. This makes the execution flow simple and easy to understand.

The design follows basic digital logic principles. Combinational units like the ALU, adders, and multiplexers compute results based only on inputs, while sequential elements like registers and memories store values and update on the clock edge.

We implemented a subset of the RV64I instruction set, including R-type, I-type, S-type, and B-type formats. Separate instruction and data memories are used to simplify the datapath. Control signals are generated using the instruction opcode and function fields to guide data movement inside the processor.

This structured approach keeps the design modular, clear, and functionally correct.

III. SYSTEM OVERVIEW

The overall organization of the processor is shown in Figure-2. The architecture consists of interconnected hardware modules that collectively implement instruction execution.

The processor is organized around clearly defined hardware modules, each performing a specific function.

The processor consists of the following primary modules:

Program Counter (PC): Stores the address of the current instruction and provides it to Instruction Memory for fetching.

Instruction Memory: Stores the program instructions and outputs a 32-bit instruction corresponding to the PC address.

Register File: Contains 32 general-purpose 64-bit registers and supplies source operands while supporting write-back of results.

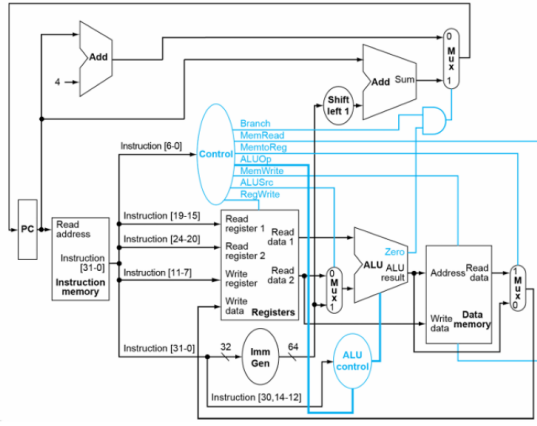


Fig. 2. Single-Cycle RV64I Processor Datapath

Main Control Unit: Decodes the instruction opcode and generates control signals that guide datapath operations.

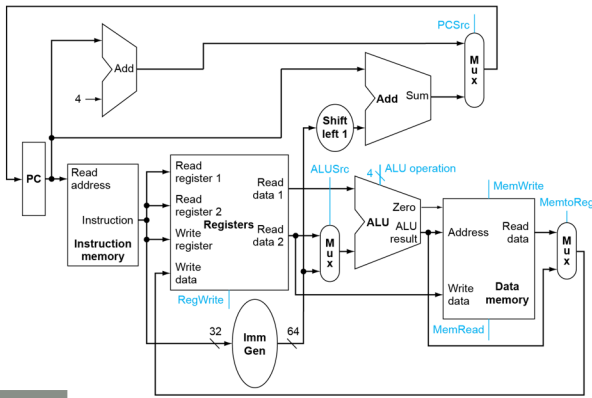
Immediate Generator: Extracts and sign-extends immediate values from different instruction formats.

ALU Control and ALU: Determine and perform the required arithmetic or logical operation based on instruction type.

Data Memory: Handles memory read and write operations for load and store instructions.

These modules work together to implement instruction execution. Detailed internal behavior of each module is described in the next section.

IV. MODULE DESCRIPTION



A. Program Counter (PC)

The Program Counter (PC) is a 64-bit register that stores the address of the current instruction. On every rising clock edge, it updates to the next instruction address. If the reset signal is active, the PC is initialized to zero to begin execution from the first instruction.

For normal execution, the next address is calculated as $PC + 4$, since each instruction is 32 bits (4 bytes). For branch

instructions, the PC is updated with the computed branch target when the Branch signal and ALU Zero flag are both asserted. A multiplexer selects between $PC + 4$ and the branch target to ensure correct control flow.

B. Instruction Memory

Instruction Memory stores the program in a byte-addressed format. Four consecutive bytes together form one 32-bit instruction. The PC provides the address input, and the corresponding instruction is fetched during the Instruction Fetch stage.

The memory contents are initialized from the `instructions.txt` file at the start of simulation. A minimum memory size of 4096 bytes is implemented as specified in the project requirements.

C. Register File

The Register File consists of 32 general-purpose registers, each 64 bits wide. It has two read ports and one write port. During the Decode stage, registers `rs1` and `rs2` are read simultaneously.

During Write Back, the result is written to register `rd` when `RegWrite` is enabled. Register `x0` is permanently hardwired to zero. Read operations are combinational, while writes occur on the clock edge.

D. Main Control Unit

The Control Unit decodes the 7-bit opcode and generates control signals including `RegWrite`, `MemRead`, `MemWrite`, `MemtoReg`, `ALUSrc`, `Branch`, and `ALUOp`. These signals determine data movement within the datapath and activate only the required hardware components for each instruction type.

E. Immediate Generator

The Immediate Generator extracts immediate values from I-type, S-type, and B-type instructions. Since immediate bits are placed differently in each format, the required fields are rearranged and sign-extended to 64 bits. The generated immediate is used for arithmetic operations, memory address computation, and branch target calculation.

F. ALU Control Unit

The ALU Control Unit determines the exact operation performed by the ALU. It combines the `ALUOp` signal from the Control Unit with the `funct3` and `funct7` fields of the instruction to produce the final ALU control code.

G. Arithmetic Logic Unit (ALU)

The ALU performs arithmetic and logical operations on two 64-bit operands. Depending on the control signal, it executes addition, subtraction, AND, OR, XOR, and shift operations. For branch instructions, the ALU performs subtraction and sets the Zero flag when both operands are equal.

The mapping between ALU control codes and operations implemented in this design is summarized below.

TABLE I
ALU OPERATION ENCODING

ALU Control	Operation
0000	ADD
0001	SUB
0010	AND
0011	OR
0100	XOR
0101	Shift Right (Logical)
0110	Shift Left (Logical)

TABLE II
MEMORY ACCESS BEHAVIOR

Instruction	MemRead	MemWrite	Operation
Load (ld)	1	0	Read 64-bit data from memory
Store (sd)	0	1	Write 64-bit data to memory
Other Instructions	0	0	No memory access

H. Data Memory

Data Memory supports 64-bit load and store operations. For load instructions, it reads data from the computed address and forwards it to the write-back stage. For store instructions, it writes register data into memory at the calculated address. Memory access is enabled only when MemRead or MemWrite is asserted.

V. STAGE-LEVEL WORKING OF THE SEQUENTIAL PROCESSOR

Even though the processor is implemented as a single-cycle design, its operation can still be understood in five logical stages. In our implementation, all these stages complete within one clock cycle, but conceptually they represent different responsibilities in instruction processing.

A. Instruction Fetch (IF)

The Instruction Fetch stage begins with the Program Counter providing the address of the next instruction. This address is sent to the Instruction Memory, which returns a 32-bit machine instruction.

At the same time, the next PC value is computed. Under normal execution, the next address is $PC + 4$. If a branch instruction is taken, the PC is updated with the branch target address computed during execution.

Thus, the IF stage is responsible for supplying a valid instruction and preparing the next instruction address.

B. Instruction Decode (ID)

In the Decode stage, the fetched instruction is divided into its fields such as opcode, rs1, rs2, rd, funct3, and funct7. The Control Unit interprets the opcode and generates the necessary control signals for execution.

Simultaneously, the Register File reads the values of the source registers. If the instruction contains an immediate value, the Immediate Generator extracts and sign-extends it to 64 bits based on the instruction format (I-type, S-type, or B-type).

This stage ensures that all required operands and control signals are ready before execution.

C. Execute (EX)

The Execute stage performs the core computation. The ALU receives two operands either two register values or one register value and one immediate depending on the ALUSrc control signal.

Based on the ALU control logic, the required operation such as addition, subtraction, AND, OR, or comparison is performed.

For load and store instructions, the ALU computes the effective memory address. For branch instructions, it compares the operands and determines whether the branch condition is satisfied.

D. Memory Access (MEM)

In this stage, memory is accessed only if required. For load instructions, the data stored at the computed memory address is read and forwarded for write-back.

For store instructions, the value from the register file is written into memory at the address calculated by the ALU.

Arithmetic and logical instructions bypass this stage since they do not require memory access.

E. Write Back (WB)

The final stage writes the result into the destination register. A multiplexer selects between the ALU result and the data read from memory.

The Register File updates the destination register only when the RegWrite signal is enabled. After this step, the instruction is considered fully executed.

VI. INSTRUCTION EXECUTION FLOW

This processor supports the following instructions: add, sub, addi, and, or, ld, sd, and beq. Each instruction activates a specific combination of control signals and datapath components.

R-Type Instructions (add, sub, and, or)

R-type instructions use two source registers (rs1, rs2) and write the result to a destination register (rd). The Control Unit sets RegWrite = 1, ALUSrc = 0, and disables memory access.

The ALU Control uses the funct3 and funct7 fields to determine whether the operation is addition, subtraction, AND, or OR. The ALU performs the operation on register operands, and the result is written back to rd.

I-Type Instruction (addi)

The addi instruction adds a sign-extended immediate value to register rs1. The Immediate Generator extracts the 12-bit immediate and extends it to 64 bits.

Here, ALUSrc = 1 selects the immediate as the second ALU operand. The ALU performs addition, and the result is written to rd. Memory access remains disabled.

Load Instruction (ld)

The `ld` instruction loads 64-bit data from memory into a register. The ALU first computes the effective address by adding `rs1` and the sign-extended immediate offset.

The Control Unit enables `MemRead` and `MemtoReg`. The data read from memory is selected through the `MemtoReg` multiplexer and written back to `rd`. This instruction activates the longest datapath in the processor.

Store Instruction (sd)

The `sd` instruction writes register data into memory. The ALU computes the effective address using `rs1` and the immediate offset.

The Control Unit enables `MemWrite` and disables `RegWrite`. The value from `rs2` is written into Data Memory at the calculated address.

Branch Instruction (beq)

The `beq` instruction compares registers `rs1` and `rs2`. The ALU performs subtraction and checks the Zero flag.

The Immediate Generator extracts and sign-extends the branch offset. This offset is shifted appropriately and added to the current PC to compute the branch target address.

If `Branch = 1` and the Zero flag is set, the PC is updated with the branch target. Otherwise, execution continues with `PC + 4`.

VII. CONTROL SIGNAL SUMMARY

The Control Unit generates a set of signals that determine how data moves through the datapath. These signals vary depending on the instruction type.

RegWrite: This signal enables writing into the Register File. It is set to 1 for instructions that produce a result (`add`, `sub`, `addi`, `and`, `or`, `ld`) and set to 0 for `sd` and `beq`.

ALUSrc: This signal selects the second input to the ALU. When `ALUSrc = 0`, the second operand comes from register `rs2` (R-type and `beq`). When `ALUSrc = 1`, the operand comes from the sign-extended immediate (I-type and load/store instructions).

MemRead and MemWrite: `MemRead` is enabled only for `ld` instructions to read data from memory. `MemWrite` is enabled only for `sd` instructions to store data into memory. For all other instructions, both signals remain 0.

MemtoReg: This signal selects the value written back to the register. When `MemtoReg = 0`, the ALU result is written. When `MemtoReg = 1`, the data read from memory is written (only for `ld`).

Branch: This signal is asserted only for `beq`. It works together with the ALU Zero flag to determine whether the PC should update to the branch target.

ALUOp: `ALUOp` is a 2-bit signal that indicates the type of operation required. For load/store instructions, `ALUOp` selects addition (address calculation). For branch instructions, it selects subtraction (comparison). For R-type instructions, `ALUOp` allows the ALU Control Unit to use `funct3` and `funct7` to determine the exact operation.

Together, these control signals ensure that each instruction activates only the required datapath components and follows the correct execution flow.

For clarity, the behavior of major control signals for each instruction type is summarized below.

Instruction	RegWrite	MemRead	MemWrite	ALUSrc
R-type	1	0	0	0
Load (ld)	1	1	0	1
Store (sd)	0	0	1	1
Branch (beq)	0	0	0	0

VIII. SINGLE-CYCLE TIMING AND CRITICAL PATH

In a single-cycle processor, every instruction completes within one clock period. This means the clock cycle must be long enough to accommodate the slowest instruction in the datapath.

Among all supported instructions, the `ld` (load) instruction activates the longest execution path. For a load instruction, the instruction is fetched from Instruction Memory, registers are read from the Register File, the ALU computes the effective address, Data Memory is accessed, and finally the data is written back to the Register File. Since all these operations must complete within one cycle, the clock period is determined by this longest path.

Other instructions such as `add` or `and` do not access Data Memory, and therefore require fewer hardware components. However, in a single-cycle design, the clock period cannot be shortened for simpler instructions. All instructions use the same fixed clock period.

This approach simplifies control logic but reduces performance efficiency. Faster designs, such as pipelined processors, improve throughput by dividing instruction execution into multiple stages across different clock cycles.

IX. VERIFICATION METHODOLOGY

To ensure correctness of the sequential RV64I processor, verification was performed in multiple stages: module-level testing, integrated processor testing, and full program execution.

Module-Level Testing

Each hardware module was tested independently using small Verilog testbenches. The Program Counter was verified for correct reset behavior and proper PC updates. The Register File was tested for correct read and write operations, including ensuring that register `x0` remains zero. The ALU was tested for all supported operations, and the Immediate Generator was verified for correct sign extension and bit extraction.

Testing modules separately helped isolate errors before integrating the full datapath.

After verifying individual modules, all components were connected to form the complete single-cycle processor. A top-level testbench generated the clock signal, applied an initial reset pulse, and initialized Instruction Memory using the `instructions.txt` file.

During simulation, one instruction was executed per clock cycle. Key signals such as PC value, ALU output, memory access signals, and control signals were monitored to ensure correct datapath behavior.

Program Execution and Output Verification

Multiple RISC-V assembly programs were executed to test arithmetic, logical, load/store, and branch instructions. At the end of execution, the processor generated a `register_file.txt` output containing the final values of all 32 registers along with the total number of clock cycles.

The output values were manually verified against expected results to confirm correct instruction execution. Waveforms were also inspected using GTKWave to debug and validate internal signal transitions.

X. POSSIBLE IMPROVEMENTS AND FUTURE WORK

While the sequential single-cycle design is simple and effective for understanding processor fundamentals, several improvements can enhance performance and functionality.

1. Pipelining: The most significant improvement would be converting the single-cycle design into a pipelined architecture. By dividing instruction execution into multiple stages operating in parallel, overall throughput can be increased significantly.

2. Hazard Handling Mechanisms: In a pipelined version, data, control, and structural hazards must be handled using techniques such as forwarding, stalling, and branch prediction. Implementing these mechanisms would make the processor more practical.

3. Expanded Instruction Support: Currently, only a subset of RV64I instructions is implemented. Extending support to additional arithmetic, logical, and jump instructions would improve completeness.

4. Performance Optimization: Critical path delay could be reduced by optimizing memory access timing and ALU design. Multi-cycle implementations may also reduce clock period compared to a pure single-cycle architecture.

5. Cache Integration: Adding instruction and data caches would significantly improve memory access performance and bring the design closer to real-world processors.

6. Exception and Interrupt Handling: Support for exceptions and interrupts would allow the processor to handle runtime events and improve robustness.

Overall, these improvements would transition the processor from a purely educational model to a more performance-oriented and feature-complete architecture.

XI. CONCLUSION

This project successfully implemented a sequential single-cycle RV64I RISC-V processor using Verilog. The processor supports arithmetic, logical, load/store, and branch instructions, and follows the standard datapath structure described in computer architecture textbooks.

Through modular design, each component such as the Program Counter, Register File, Control Unit, ALU, and Data Memory was developed and verified independently before full integration. The final processor correctly executed multiple RISC-V assembly programs and produced accurate register outputs.

Although the single-cycle design is simple and easy to understand, it highlights an important limitation: the clock period is determined by the slowest instruction. This project therefore provides a strong foundation for understanding more advanced designs such as pipelined processors, where performance can be improved by overlapping instruction stages.

Overall, this implementation strengthened our understanding of how instructions are executed at the hardware level and how control and datapath components work together to form a complete processor.

REFERENCES

[1] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface, RISC-V Edition*, Morgan Kaufmann, 2018.

XII. TEAM CONTRIBUTIONS

The project was developed collaboratively with responsibilities distributed across hardware module design, integration, and verification.

Naga Rama Hari Kumar (2024102045) - Designed and implemented the ALU Control Unit - Implemented the Data Memory module - Contributed to verification of arithmetic and memory operations

Veekshita Sai (2024102026) - Designed and implemented the Immediate Generator - Implemented the Instruction Memory module - Designed the Program Counter (PC) module

Chandini Gayathri (2024102020) - Integrated all modules into the top-level Processor - Implemented the Register File - Developed the final sequential testbench (`seq_tb`)

Collective Contribution All team members jointly worked on datapath integration, control signal validation, debugging, waveform analysis using GTKWave, testcase execution, performance verification, and report preparation. Final system-level testing and documentation were completed collaboratively to ensure correctness and completeness.

XIII. RESULTS

```

=====
REGISTER FILE TESTBENCH
=====
Write x1=100 | Read x1=100 (Exp:100) | Read x0=0 (Exp:0) | PASS
Write x2=-5 | Read x2=-5 (Exp:-5) | PASS
Write x0=99 | Read x0=0 (Exp:0, hardwired) | PASS
Read x1=100 x2=-5 simultaneously | PASS
=====
GTKWave: gtkwave gtk_register_file.vcd
=====
register file tb.v:69: $finish called at 38 (1s)
j-chandini-gayathri@j-chandini-gayathri-Victus-by-HP-Gaming-Laptop-1

```

Fig. 3.

```

VCD info: dumpfile gtk_alu.vcd opened for output.
=====
ALU TESTBENCH
=====
Op | input1 | input2 | result | zero | Status
---|---|---|---|---|---
ADD | in1=000000000000000f | in2=000000000000000a | result=0000000000000019 | zero=0 | PASS
ADD | in1=0000000000000000 | in2=0000000000000000 | result=0000000000000000 | zero=1 | PASS
ADD | in1=ffffffffffffff | in2=0000000000000001 | result=0000000000000000 | zero=1 | PASS
SUB | in1=00000000000000014 | in2=0000000000000005 | result=000000000000000f | zero=0 | PASS
SUB | in1=0000000000000005 | in2=0000000000000005 | result=0000000000000000 | zero=1 | PASS
SUB | in1=0000000000000005 | in2=000000000000000a | result=ffffffffffffffb | zero=0 | PASS
AND | in1=000000000000000f | in2=000000000000000f | result=000000000000000f | zero=0 | PASS
AND | in1=00000000000000ff | in2=0000000000000000 | result=0000000000000000 | zero=1 | PASS
OR | in1=00000000000000f0 | in2=000000000000000f | result=00000000000000ff | zero=0 | PASS
OR | in1=0000000000000000 | in2=0000000000000000 | result=0000000000000000 | zero=1 | PASS
=====
GTKWave: gtkwave gtk_alu.vcd
=====
alu tb.v:63: $finish called at 100 (1s)

```

Fig. 4.

```

VCD info: dumpfile gtk_alu_control.vcd opened for output.
=====
ALU CONTROL TESTBENCH
=====
Instr | ALUOp funct3 funct7 | ctrl | Status
---|---|---|---|---
DD(ld) | ALUOp=00 funct3=000 funct7=0000000 | alu_ctrl=0000 | PASS
B(beq) | ALUOp=01 funct3=000 funct7=0000000 | alu_ctrl=1000 | PASS
ADD(R) | ALUOp=10 funct3=000 funct7=0000000 | alu_ctrl=0000 | PASS
SUB(R) | ALUOp=10 funct3=000 funct7=0100000 | alu_ctrl=1000 | PASS
OR(R) | ALUOp=10 funct3=110 funct7=0000000 | alu_ctrl=0110 | PASS
AND(R) | ALUOp=10 funct3=111 funct7=0000000 | alu_ctrl=0111 | PASS
=====
GTKWave: gtkwave gtk_alu_control.vcd
=====
alu control tb.v:48: $finish called at 60 (1s)

```

Fig. 5.

```

out
VCD info: dumpfile gtk_control_unit.vcd opened for output.
=====
CONTROL UNIT TESTBENCH
=====
Instr | opcode | RW MR MW Br AS M2R ALUOp | Status
---|---|---|---|---|---
R-type | opcode=0110011 | RW=1 MR=0 MW=0 Br=0 AS=0 M2R=0 ALUOp=10 | PASS
addi | opcode=0010011 | RW=1 MR=0 MW=0 Br=0 AS=1 M2R=0 ALUOp=00 | PASS
ld | opcode=0000011 | RW=1 MR=1 MW=0 Br=0 AS=1 M2R=1 ALUOp=00 | PASS
sd | opcode=0100011 | RW=0 MR=0 MW=1 Br=0 AS=1 M2R=0 ALUOp=00 | PASS
beq | opcode=1100011 | RW=0 MR=0 MW=0 Br=1 AS=0 M2R=0 ALUOp=01 | PASS
=====
GTKWave: gtkwave gtk_control_unit.vcd
=====
control unit tb.v:54: $finish called at 50 (1s)

```

Fig. 6.

```

out
VCD info: dumpfile gtk_data_memory.vcd opened for output.
=====
DATA MEMORY TESTBENCH (Big-Endian 64-bit)
=====
Write 0xDEADBEEFCAFEBABE @ addr=0
Read back = deadbeefcafebabE | PASS
Write 15 @ addr=8 | Read=15 (Exp:15) | PASS
Write -5 @ addr=16 | Read=-5 (Exp:-5) | PASS
Big-endian check: mem[0]=de (Exp:DE) | PASS
=====
GTKWave: gtkwave gtk_data_memory.vcd
=====
data memory tb.v:61: $finish called at 28 (1s)

```

Fig. 7.

```

out
VCD info: dumpfile gtk_immediate_gen.vcd opened for output.
=====
IMMEDIATE GEN TESTBENCH
=====
instr=00500093 | imm=0000000000000005 (5) | PASS
instr=ffff0093 | imm=ffffffffffffff (-1) | PASS
instr=7ff00093 | imm=00000000000007ff (2047) | PASS
instr=00120223 | imm=0000000000000018 (24) | PASS
instr=00520463 | imm=0000000000000008 (8) | PASS
instr=fe520ce3 | imm=ffffffffffffff (-8) | PASS
=====
GTKWave: gtkwave gtk_immediate_gen.vcd
=====
immediate gen tb.v:48: $finish called at 60 (1s)

```

Fig. 8.

```

=====
Addr | Instr | Decoded opcode
-----|---|---
PC=0 | 00a00093 | opcode=0010011 (addi)
PC=4 | 00000193 | opcode=0010011 (addi)
PC=8 | 00100213 | opcode=0010011 (addi)
PC=12 | 00000463 | opcode=1100011 (sub)
PC=48 | 00000000 | opcode=0000000 (beq)
PC=56 | 00000000 | opcode=0000000 (add)
PC=60 | 00000000 | (past end, Exp:0000000) | PASS
=====
GTKWave: gtkwave gtk_instruction_mem.vcd
=====
instruction mem tb.v:35: $finish called at 70 (1s)

```

Fig. 9.

```

VCD info: dumpfile gtk_pc.vcd opened for output.
=====
PC TESTBENCH
=====
After reset | pc_out=0 (Exp:0) | PASS
PC+4 (4) | pc_out=4 (Exp:4) | PASS
PC+4 (8) | pc_out=8 (Exp:8) | PASS
Branch (48) | pc_out=48 (Exp:48) | PASS
PC+4 (52) | pc_out=52 (Exp:52) | PASS
Reset again | pc_out=0 (Exp:0) | PASS
=====
GTKWave: gtkwave gtk_pc.vcd
=====
pc tb.v:56: $finish called at 56 (1s)

```

Fig. 10.

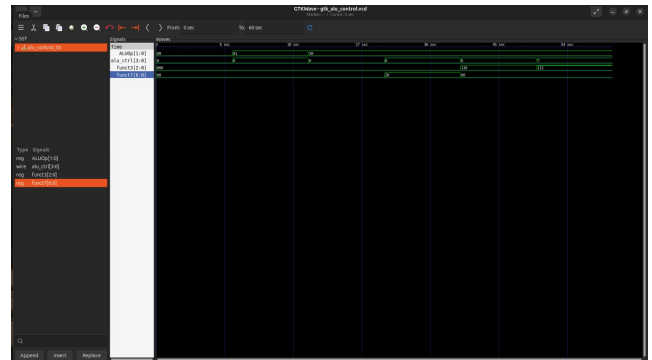


Fig. 11. ALU Control

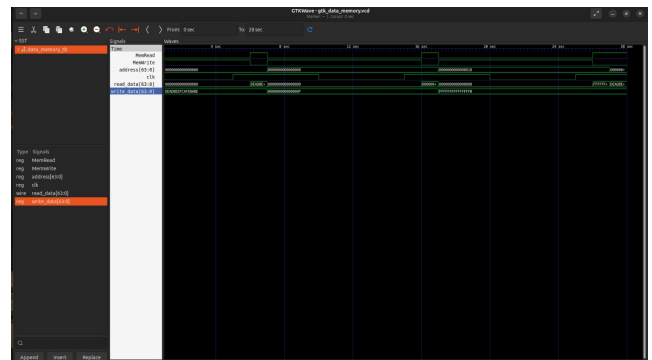


Fig. 12. Data Memory

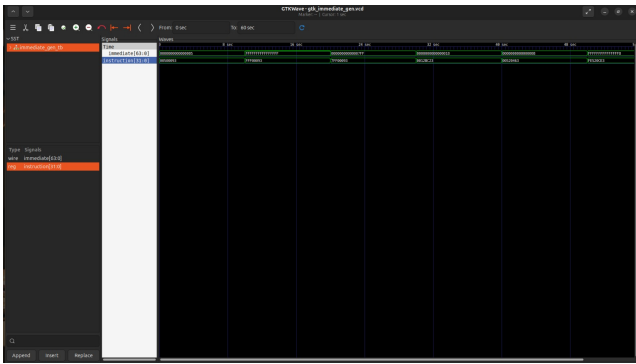


Fig. 13. Immediate Gen

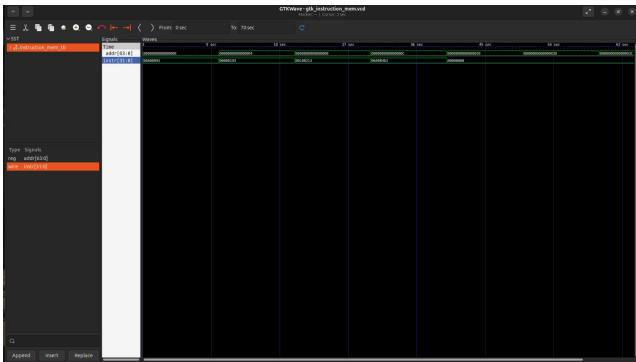


Fig. 14. Instruction Memory

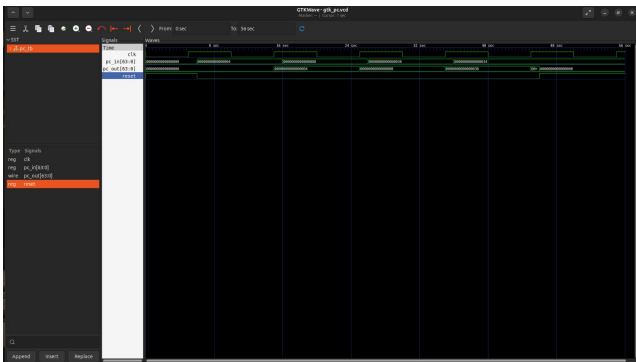


Fig. 15. PC

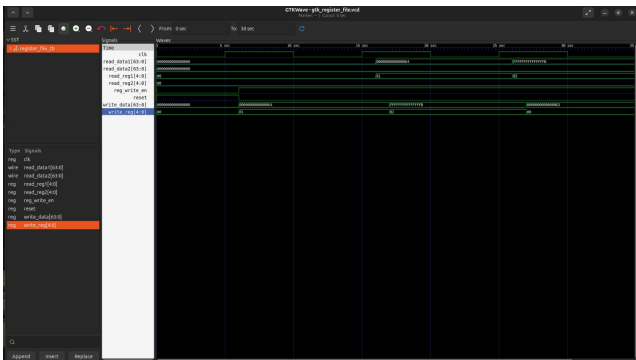


Fig. 16. Register File